Binaml: Continual Learning over Binary Feature Streams

Romain Zimmer
hi@romainzimmer.com

2026-08-11 (Draft)

Abstract

Binaml focuses on continual learning models over streams of binary features subject to data drift. Binary features provide a task-agnostic representation and enable models optimized for memory, latency, and energy efficiency. This paper specifies Binaml's online ensemble models, `BRegressor` for scalar regression and `BClassifier` for multiclass classification, as ensembles of batch-learned boolean functions with task-specific linear heads. Both models adapt online rather than relying on a stationary train/serve assumption and learn compact compositional representations from residual feedback. The models are environment-agnostic: they do not depend on how the stream is produced. This paper specifies the current models and reports reproducible prequential comparisons on versioned synthetic regression and classification streams.

1 Notation

d, x_t Input width and binary input.

$y_t, \hat{y}_t$ Scalar regression target and prediction.

$K, c_t, \hat{c}_t$ Number of classes, class label, and predicted class (classification).

e_t, s_t Residual and residual-sign label.

b, w_k, η, λ Intercept, ensemble weight of function k, learning rate, and ℓ_2 decay.

$f_k(x)$ Stored boolean function k mapping $\{0,1\}^d$ to $\{0,1\}$, stored as a compact boolean function.

$T \in \{0, \dots, 15\}$ Four-bit truth table of an arity-2 composed node.

$B, S, n, K, L_{\max}, K_{\max}$ Feature batch size, SGD steps per observation, batch index, parent top-K width, maximum builder depth, and ensemble capacity.

2 Model

Both `BRegressor` and `BClassifier` share the same boolean function discovery pipeline: batch-local graph growth, in-sample output selection, compaction into `FunctionGraph` objects, and ensemble pruning at capacity $K_{\max}$. They differ only in the linear head that maps stored function activations to predictions.

2.1 Regression head

At time t, the regressor receives $x_t \in \{0,1\}^d$ and later observes a finite scalar y_t. If useful signal is sparse weighted boolean structure over bits, a linear head over learned boolean indicators is a direct hypothesis class: continuous amplitudes, discrete features, and an inspectable composition. Let K_t be the number of stored functions after batch finalizations through time t. The prediction is

$$\hat{y}_t = b + \sum_{k=1}^{K_t} w_k \, f_k(x_t),$$

where $b \in \mathbb{R}$, each $w_k \in \mathbb{R}$, and each $f_k(x_t) \in \{0,1\}$. Each f_k is a compact boolean function built from one residual-sign batch. Feature learning consumes a binary supervision bit derived from the scalar target. With pre-update prediction $\hat{y}_t$, the residual is $e_t = y_t - \hat{y}_t$ and the sign appended to the current feature batch is

$$s_t = \mathbf{1}\{e_t \geq 0\}.$$

This bit is computed with the current head parameters before the linear updates on step t.

Weight updates. For one SGD step with learning rate $\eta > 0$ and decay $\lambda \geq 0$, first apply ℓ_2 decay to every stored weight, then update the intercept and weights from the pre-step residual:

$$\begin{aligned} w_k &\leftarrow (1 - \eta\lambda)\, w_k && \text{for all stored } k, \\ b &\leftarrow b + \eta e_t, \\ w_k &\leftarrow w_k + \eta e_t f_k(x_t). \end{aligned}$$

Each observation performs S such steps on the same activation row.

2.2 Classification head

The classifier receives $x_t \in \{0,1\}^d$ and later observes a class label $y_t \in \{0, \ldots, K-1\}$. Let K_t be the number of stored functions after batch finalizations through time t. For each class c, logits are

$$\ell_{t,c} = b_c + \sum_{k=1}^{K_t} w_{k,c} \, f_k(x_t),$$

where $b_c \in \mathbb{R}$ and each $w_{k,c} \in \mathbb{R}$. The predicted class is $\hat{c}_t = \operatorname{argmax}_c \ell_{t,c}$, with ties broken toward the smallest index. Feature learning consumes a binary supervision bit derived from the class label. With pre-update prediction $\hat{c}_t$, the sign appended to the current feature batch is the misclassification indicator

$$s_t = \mathbf{1}\{\hat{c}_t \neq y_t\}.$$

This bit is computed with the current head parameters before the linear updates on step t.

Weight updates. Let $p_{t,c} = \operatorname{softmax}(\ell_t)_c$ be the pre-step class probabilities. For one SGD step on softmax cross-entropy with learning rate $\eta > 0$ and decay $\lambda \geq 0$, define the class errors $\delta_{t,c} = p_{t,c} - \mathbf{1}\{c = y_t\}$. First apply ℓ_2 decay to every stored weight, then update intercepts and weights:

$$\begin{aligned} w_{k,c} &\leftarrow (1 - \eta\lambda)\, w_{k,c} && \text{for all stored } k, c, \\ b_c &\leftarrow b_c - \eta \, \delta_{t,c}, \\ w_{k,c} &\leftarrow w_{k,c} - \eta \, \delta_{t,c} \, f_k(x_t). \end{aligned}$$

Each observation performs S such steps on the same activation row.

3 Building stored functions

Composition starts from the most basic nontrivial boolean learning unit: two-input tables. Combining them yields richer features without beginning from high-arity search. A composed node combines two parent activations using a four-bit truth table $T \in \{0, \dots, 15\}$.

3.1 Feature representation and batch learning

Each feature batch grows an ephemeral wide DAG: layer zero holds the d source bits; at each subsequent layer the builder forms every distinct pair among the top-K parent nodes by descending $|\text{assoc}|$, learns a truth table on the current batch, and appends the resulting node. Depth never exceeds $L_{\max}$. When the batch fills, the builder selects one output node by descending in-sample accuracy, then depth, then node id, inverting the output bit when the inverted node is more accurate on $\mathcal{B}_n$. The subgraph is compacted into a FunctionGraph and appended to the ensemble with weight zero. If the ensemble exceeds capacity $K_{\max}$, the function with smallest $|w_k|$ is removed, breaking ties toward the oldest index. There is no hold-out batch and no shared candidate queue across batches.

3.2 Compact evaluation

A stored FunctionGraph lists source indices and composed nodes in topological order. Each node is either a source coordinate, a constant, or a composed operation referencing two already evaluated positions and one four-bit truth table. One forward pass evaluates $f_k(x)$ for prediction and SGD. The eight u8 counters used to learn a table cover every $(u, v, s) \in \{0, 1\}^3$ combination, which limits the feature batch size to $B \leq 255$.

Type	Fields or representation	Role
FunctionBuildConfig	{batch_size, parent_top_k, max_layers}	batch-local graph growth limits
FunctionBuilder	stateless builder	grows ephemeral DAG from one SignBatch
FunctionGraph	sources, compact nodes, output index	immutable stored function
CompactNode	Source, Constant, or Composed{first, second, truth_table}	one evaluation step
BRegressor	functions: Vec<FunctionGraph>, weights: Vec<f64>, intercept, batch buffers, count	online regression ensemble state
BClassifier	functions: Vec<FunctionGraph>, weights: Vec<Vec<f64», intercept vector, batch buffers, count	online classification ensemble state

3.3 Residual-sign supervision and batching

Feature learning targets binary supervision bits, not the task target directly. Each head maps its target to $s_t \in \{0, 1\}$ as above; the builder treats s_t as a binary label while the linear head absorbs magnitude or class structure. Exact truth-table counting needs a bounded window; batching amortizes graph growth. Batch n is

$$\mathcal{B}_n = \{(x_t, s_t)\}_{t \in I_n}, \qquad |I_n| = B,$$

where I_n is the contiguous block of B observations since the previous structural update. Processing $\mathcal{B}_n$ grows an ephemeral graph, selects one output by in-sample accuracy, compacts it, appends $(f, w = 0)$ to the ensemble, and prunes to $K_{\max}$ if needed. There is no hold-out batch.

Association score. For a boolean stream $z_{1:B}$ and residual signs $s_{1:B}$, let $N = B$, $N_x = \sum_i z_i$, $N_y = \sum_i s_i$, and $N_{xy} = \sum_i z_i s_i$. Define

$$\operatorname{assoc}(z, s) = N\, N_{xy} - N_x N_y.$$

Parent nodes are ranked by $|\operatorname{assoc}(z, s)|$, breaking ties by node id. Sources use their coordinate columns; composed nodes use their evaluated bitstreams. Negation at layer zero is not materialized as separate nodes: anti-correlated sources rank highly on $|\operatorname{assoc}|$, and output polarity flip handles $\neg x_i$ when it improves in-sample accuracy.

3.4 Truth-table learning

Given two parent bitstreams $u_{1:B}$, $v_{1:B}$ and residual signs $s_{1:B}$, the learner returns the Hamming-error-minimizing four-bit table. Its state is a fixed array of eight `u8` counters, one for each combination of the two parent values and residual sign. The learned result is a four-bit truth table represented by one `u8`. The counters record the frequencies needed to select each table output by majority, with ties resolving to zero. As one counter may receive every observation, the `u8` counters bound the admissible batch size to $B \leq 255$.

3.5 Batch graph growth

For each layer $\ell = 1, \ldots, L_{\max}$:

1. Let P be the top-K nodes in layer $\ell - 1$ by $|\operatorname{assoc}|$, breaking ties by node id.
2. Form every distinct pair in P.
3. For each pair, learn a truth table on $\mathcal{B}_n$ and append the resulting composed node to layer ℓ.

If P has fewer than two nodes, layer ℓ is skipped. After all layers, the builder selects the output node by descending in-sample accuracy, then depth, then id, inverting the output when that improves accuracy on $\mathcal{B}_n$.

3.6 Ensemble pruning

When appending a new function would exceed $K_{\max}$, remove the stored index with smallest $|w_k|$, breaking ties toward the oldest index. Zero-weight experts can therefore rotate at capacity even when still structurally useful on past batches.

4 Online head updates

Function identity is discrete; head parameters are continuous, so the two are updated separately. For each observation, `predict` evaluates the current stored functions once; `update` then computes the head-specific supervision bit s_t, appends (x_t, s_t) to the current feature batch, and performs S single-sample gradient steps on that activation row using the task-specific rules above. In both models the newly appended function at a batch boundary is excluded from SGD on the batch-fill step because it is added only after those updates. There is no gradient through boolean structure: SGD tracks task loss at the sample level, while structure search runs when enough supervision bits have accumulated. After every B new observations, the batch is consumed to grow, select, compact, and append one function, then cleared.

5 Online learning procedure

Configuration

Linear Learning rate η, decay λ, and S SGD steps per observation.

Batch structure Feature batch size B, parent top-K width K, maximum builder depth $L_{\max}$, and ensemble capacity $K_{\max}$.

Initialize: empty ensemble, $b \leftarrow 0$, and empty feature-batch buffers.
For each observation (x_t, y_t):

1. Validate the input width and task target.
2. Evaluate stored $f_k(x_t)$ and form the task prediction ($\hat{y}_t$ or $\hat{c}_t$).
3. Compute the head-specific supervision bit s_t and append (x_t, s_t) to the current feature batch.
4. Apply S single-sample head updates using the current activation row.
5. After B accumulated observations, grow a graph on the batch, select and compact one output, append it with weight zero, prune to $K_{\max}$ if needed, and clear the feature batch.

Cost. Prediction and activation evaluation are linear in the number of stored functions and their node counts. Each linear step costs one function count. Structural work occurs once per feature batch: graph growth scans selected parent pairs over B rows, then compaction produces one stored function.

6 API and integration boundaries

The `binaml.models` package owns the online models and has no environment dependency: the goal is generic models, decoupled from where the data comes from and how it was produced. The public surfaces are `BRegressor` and `BClassifier`, which implement `predict(features)` and `update(target)` for binary feature vectors of width d. The prediction call retains pairing state; `update` must follow a preceding `predict` and applies the revealed target to that stored step. Feature learning is internal to `update` once a batch fills. Evaluation packages compose models with environments through predict-then-update prequential evaluation. Named benchmarks only compose these independent components.

7 Prequential evaluation protocol

For each emitted sample, an evaluator first records the model prediction, then exposes the target through `update`. Regression benchmarks compute mean squared error from the recorded pre-update predictions; classification benchmarks compute accuracy from the recorded pre-update class predictions. Structure updates occur only after a full feature batch fills inside `update`; residual signs are measured with the current head parameters before the corresponding updates. Hyperparameters, implementation version, and the online protocol are required to interpret any reported trajectory.

8 Evaluation

The benchmark inputs and model configurations are versioned in this paper's benchmark directory.

8.1 Regression

The regression task uses 1,000 observations and seeds $\{0, 1, 2, 3, 4\}$ with 32 input bits, 12 fixed target components, noise standard deviation 0.1, and the same function, DAG, and target-scale settings. Input, gate, and intercept distributions each resample independently with probability 0.01.

All models receive the scenario width. `BRegressor` uses learning rate $\eta = 3 \times 10^{-2}$, ℓ_2 decay 5×10^{-4}, feature batch size $B = 16$, twenty single-sample SGD steps, parent top-$K = 8$, builder depth $L_{\max} = 3$, and ensemble capacity $K_{\max} = 64$. Each stored function enters the linear head as $f_k(x) \in \{0, 1\}$. The linear baseline is a JAX replay-batch implementation using learning rate 0.03, the same decay, replay size 32, and step count, with uncentered binary inputs. The MLP baseline is a JAX one-hidden-layer ReLU network using replay size 32 and three full-batch SGD steps; one 50-unit hidden layer; constant learning rate 0.003, $\alpha = 10^{-4}$, and random state zero. Linear and MLP baselines share the same float32 replay SGD scaffold.

Tables report the mean ± standard error over five seeds. MSE is the optimized loss; RMSE expresses the same error in target units. Total time is the prequential predict-plus-update time for a 1,000-sample trajectory.

Model	MSE	RMSE	total s
BRegressor (ours)	$\mathbf{0.430 \pm 0.014}$	$\mathbf{0.656 \pm 0.011}$	$\mathbf{0.046 \pm 0.001}$
Linear SGD (JAX)	0.825 ± 0.024	0.908 ± 0.013	5.021 ± 0.063
MLP SGD (JAX)	0.907 ± 0.018	0.952 ± 0.009	6.217 ± 0.055

8.2 Classification

The classification task uses the same input width, component count, noise standard deviation, drift probabilities, and function settings as the regression default, with three classes and per-class gates, weights, and intercepts. Scenario and model entries are versioned in `benchmarks/scenarios/default_multiclass.` and `benchmarks/models_classification.json`. `BClassifier` uses learning rate $\eta = 1.2 \times 10^{-1}$, ℓ_2 decay 5×10^{-4}, feature batch size $B = 12$, thirty single-sample SGD steps, and the same structural limits as `BRegressor`; linear and MLP classifier baselines mirror their regression counterparts with softmax cross-entropy. Reported comparisons use prequential accuracy over the same seed set after optional warm-up exclusion.

Tables report the mean ± standard error over five seeds. Accuracy is the fraction of correct pre-update class predictions on the 900 post-warm-up observations. Total time is the prequential predict-plus-update time for a 1,000-sample trajectory.

Model	accuracy	total s
BClassifier (ours)	$\mathbf{0.746 \pm 0.023}$	$\mathbf{0.088 \pm 0.001}$
Linear SGD (JAX)	0.653 ± 0.038	0.796 ± 0.034
MLP SGD (JAX)	0.645 ± 0.041	1.153 ± 0.029

9 Limitations and scope

This document specifies the ensemble models used by Binaml. It does not claim optimality of residual-sign learning or in-sample function selection. Ensemble churn at capacity can drop zero-weight experts that remain useful on past batches; in-sample output selection trades hold-out promotion for simpler batch boundaries. The reported regression comparison is limited to the versioned synthetic task, fixed configurations, and reproducible baseline runs described above.